

PROGRAMACIÓN ORIENTADA A OBJETOS - HERENCIA

Curso de Programación de Algoritmos con Java
Universidad Técnica Particular de Loja – Ecuador
Wayner Bustamante– wxbustamante@utpl.edu.ec



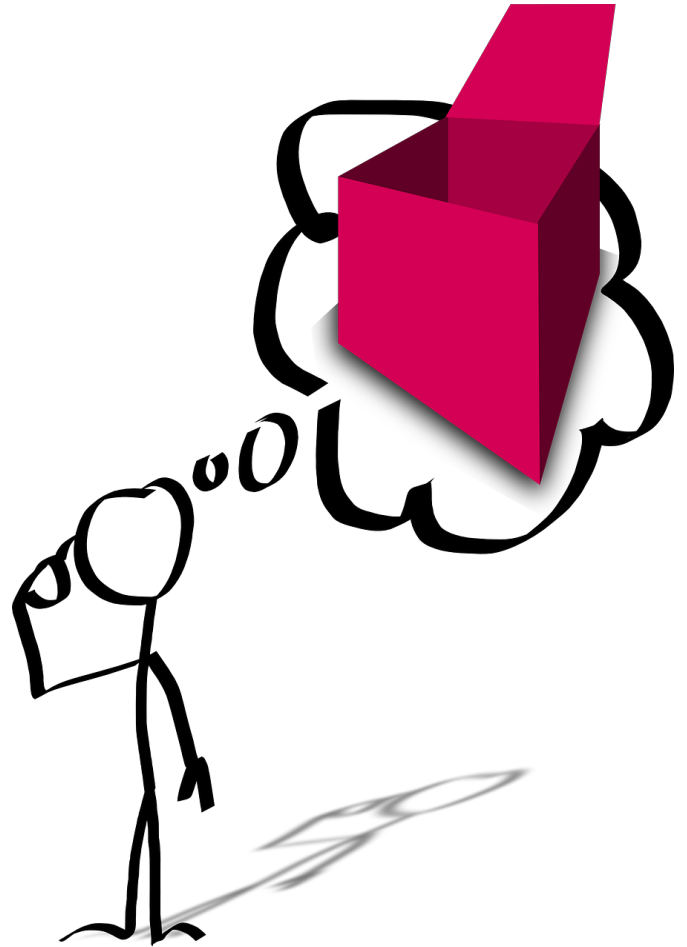
AGENDA

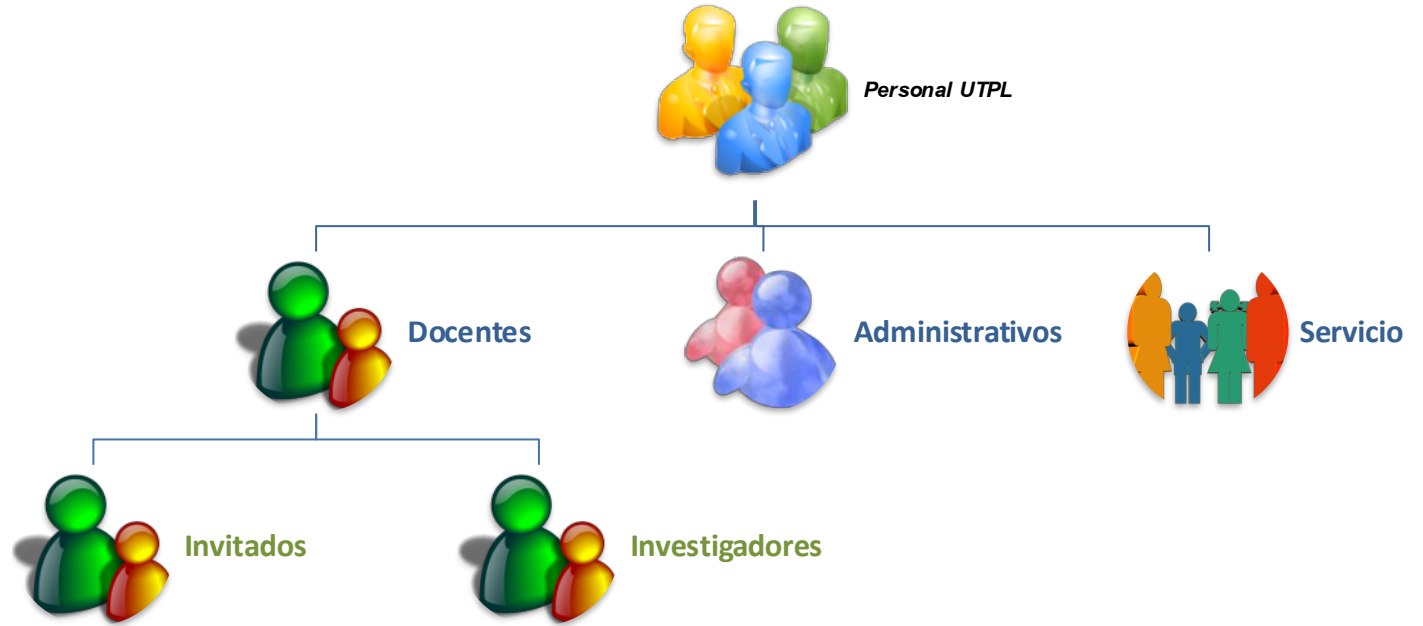
- ✓ **Que es la herencia** (*SuperClases y SubClases*).
- ✓ **Tipos de herencia** (*simple, multiple, directa, indirecta*)
- ✓ **Jerarquía de Herencia** (*niveles*)
- ✓ **Miembros public y protected** (*niveles de acceso*)
- ✓ **Palabra clave super** (*llamado a miembros y constructores*)
- ✓ **Llamando a constructores** (*implicita, explicita*)
- ✓ **Estudio de casos** (*ejercicios*)

HERENCIA

Forma de reutilizar software creando nuevas clases (*subClases*) a partir de otras (*superClases*), absorbiendo, modificando y mejorando los miembros y capacidades de sus clases padre.

- ✎ Especializa o extiende funcionalidad.
- ✎ Optimiza cantidad de código.
- ✎ Permite diseño jerárquico de clases.
- ✎ Ocultación de información. (*encapsulamiento*)



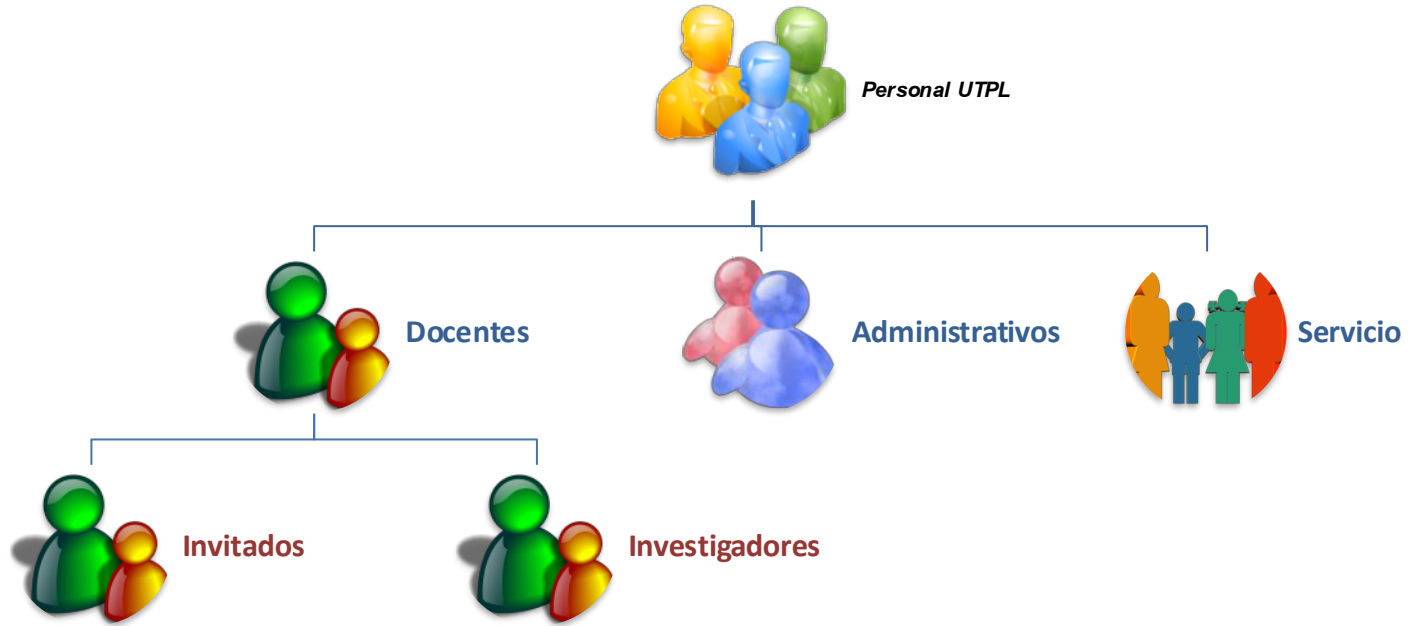


Herencia directa

“es un”

Personal UTPL ➡ Docentes, Administrativos, Servicio

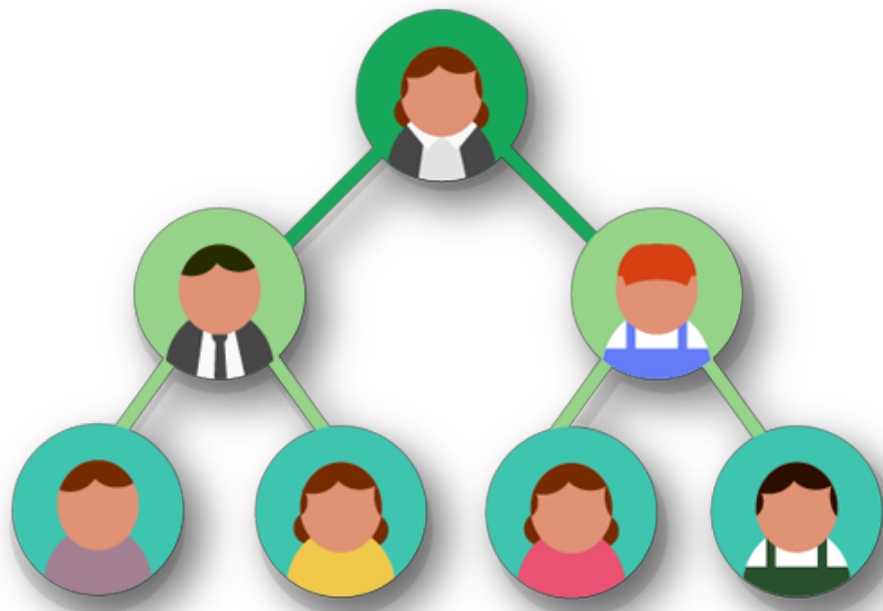
Docentes ➡ Invitados, Investigadores



Herencia indirecta

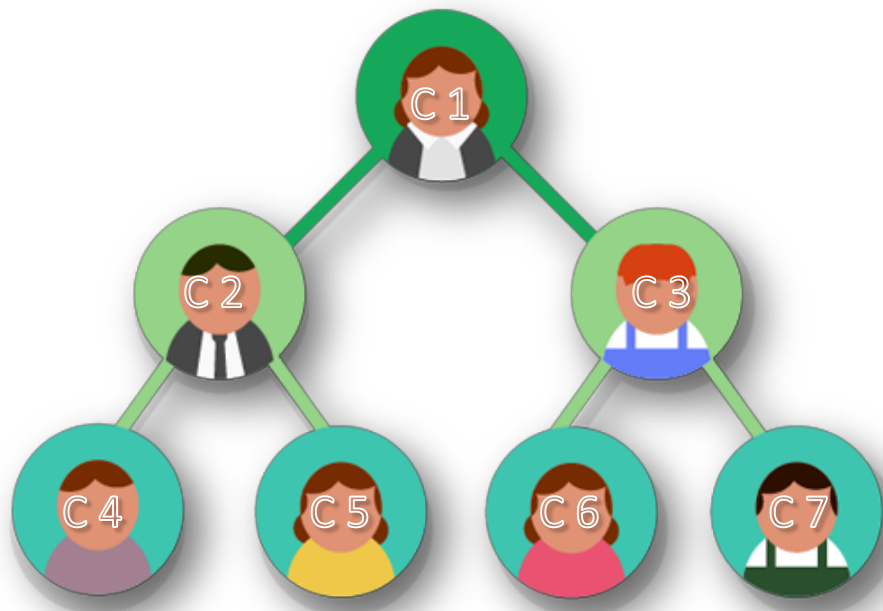
“es un”

Personal UTPL ➡ Invitados, Investigadores



Jerarquía herencia

Clases “*afiliadas*” con otras clases, donde unas son *superclase*, que proporcionan miembros a otras (*subClases*). A veces una clase es tanto superclase como subclase.

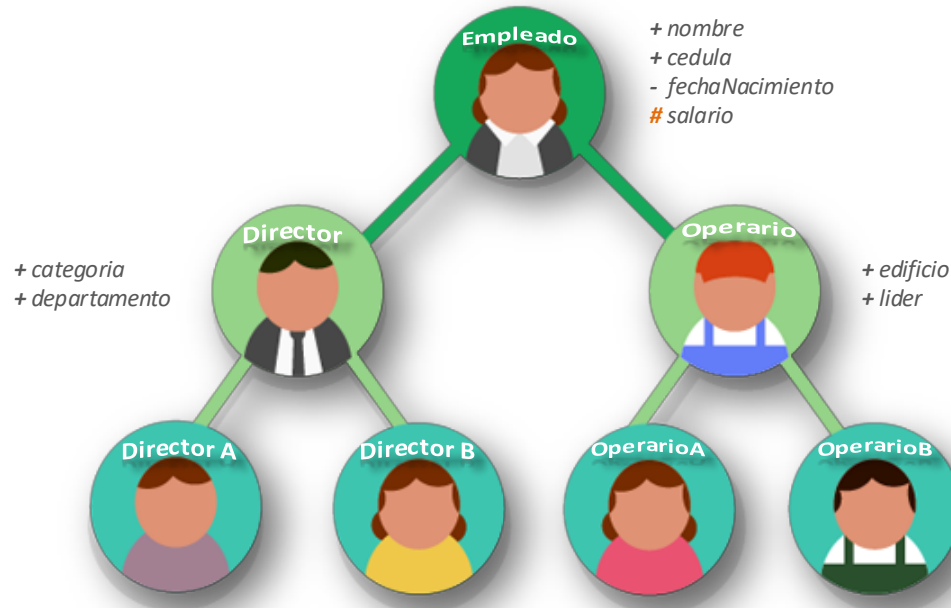


Jerarquía herencia

C1, C2 y C3 ➡ SuperClases

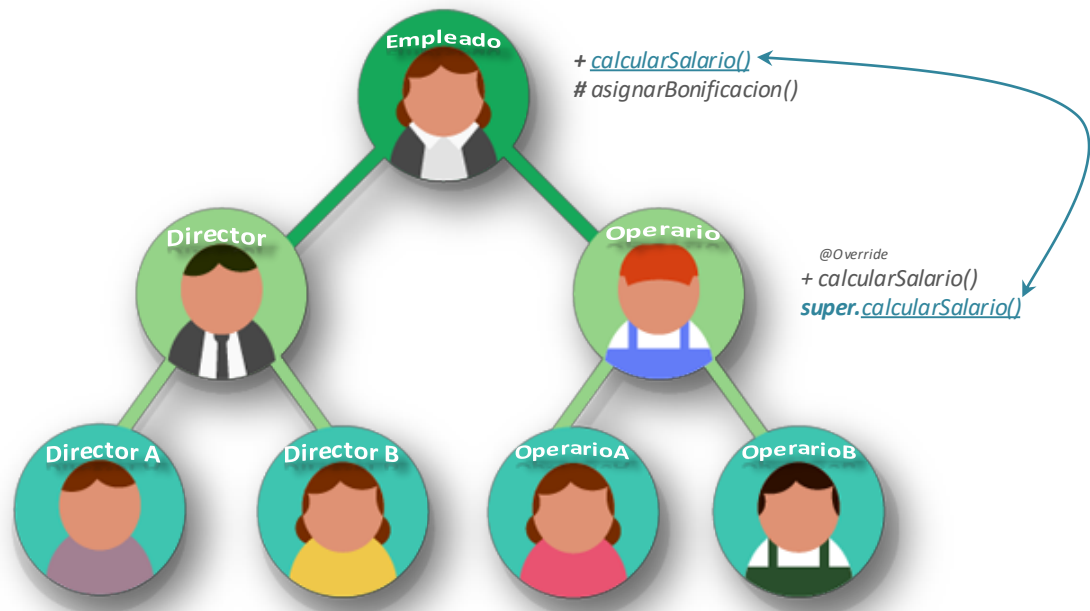
C2, C3, C4, C5, C6 y C7 ➡ SubClases

C2, C3 ➡ Super y Sub clases



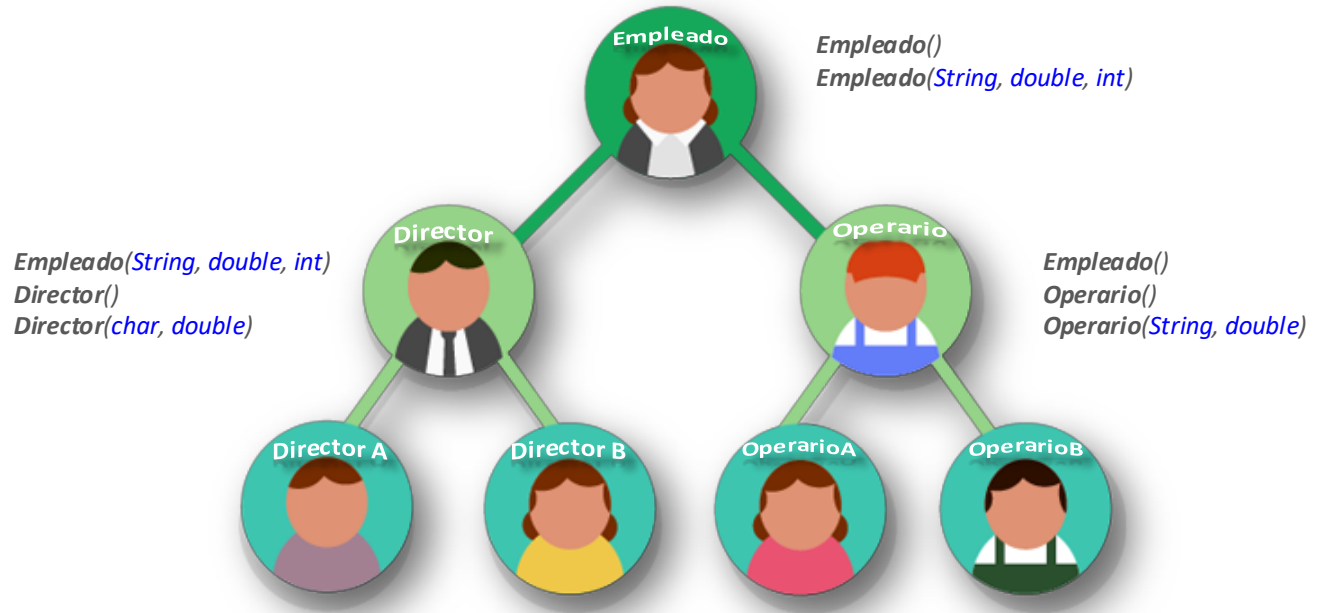
Miembros protected

- ✎ Nivel intermedio de acceso: *public* vs *protected*.
- ✎ Lo *protected* de superClase son usados por la superClase (*this*), las subClases (*super*) y otras clases del mismo paquete.
- ✎ Public y *protected* de superClase retienen su modificador de acceso original cuando se convierten en miembros de subClase (*se heredan*).



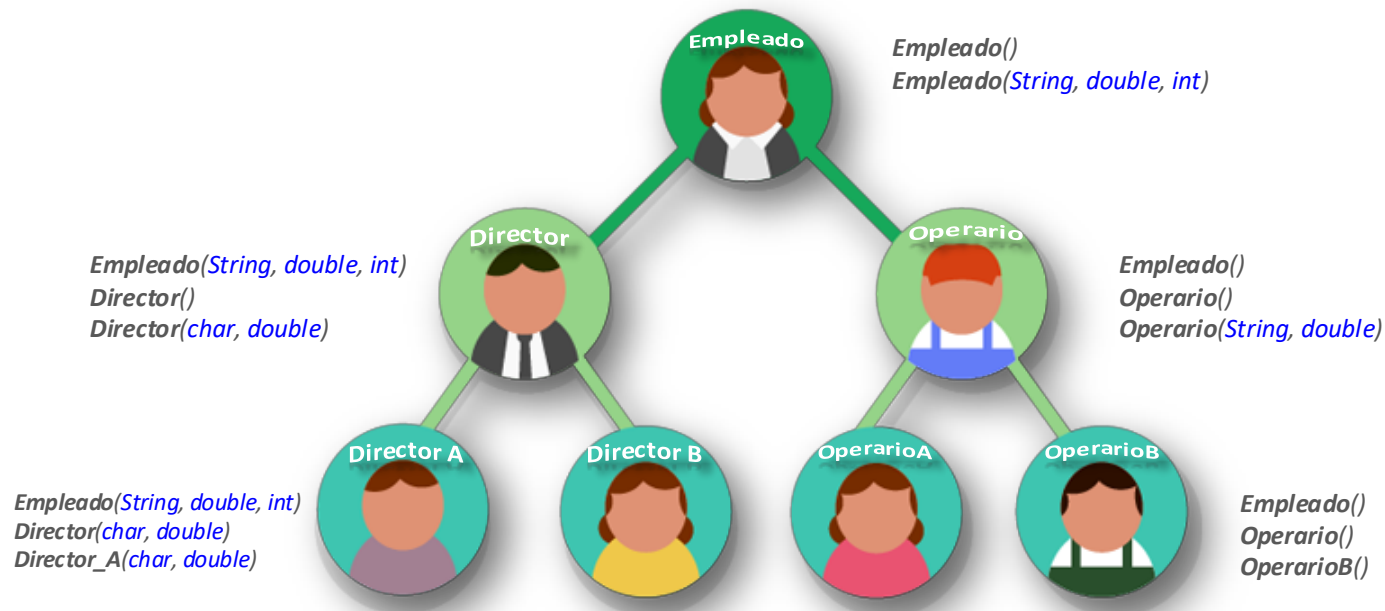
super.padre

- Los métodos de subClase citan lo public y **protected** de la superClase sólo usando sus nombres (o `this` o `super`).
- Cuando un método de la subClase sobrescribe un método de la superClase, éste último puede utilizarse desde la subClase si se antepone a su nombre la palabra clave **super** y un punto (`.`)



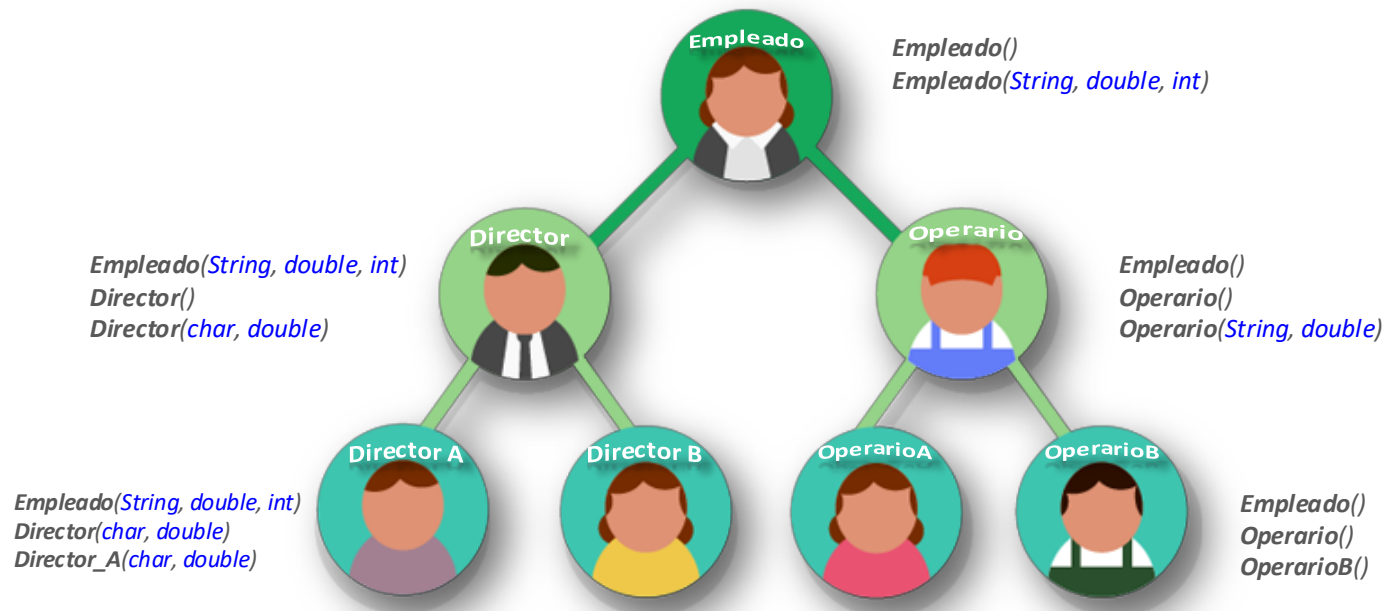
Constructores

- ✎ Al instanciar una *subClase* primero se llama a los constructores de sus *superClases*, explícita (con **super**) o implícitamente (**constructor** predeterminado o **sin argumentos**).



Constructores

- Si la superClase se deriva de otra clase, el constructor de la superClase invoca primero al constructor de la siguiente clase que se encuentre a un nivel más arriba (su clase padre).



Constructores

- ✎ El último constructor que se llama en la cadena es siempre el de la clase **Object**.
- ✎ El cuerpo del **constructor** de la **subclase** original termina de ejecutarse al último.

ESTUDIO DE CASO (1)

Defina una jerarquía de herencia en la que se contemplen los empleados de una empresa (*operarios y gerentes*), guarde los nombres y sueldo; para los gerentes como un tipo especial de empleados, almacene el departamento que dirige y el nombre de su secretaria, para operarios, horas laboradas. Realizar la especificación UML con las clases implicadas.



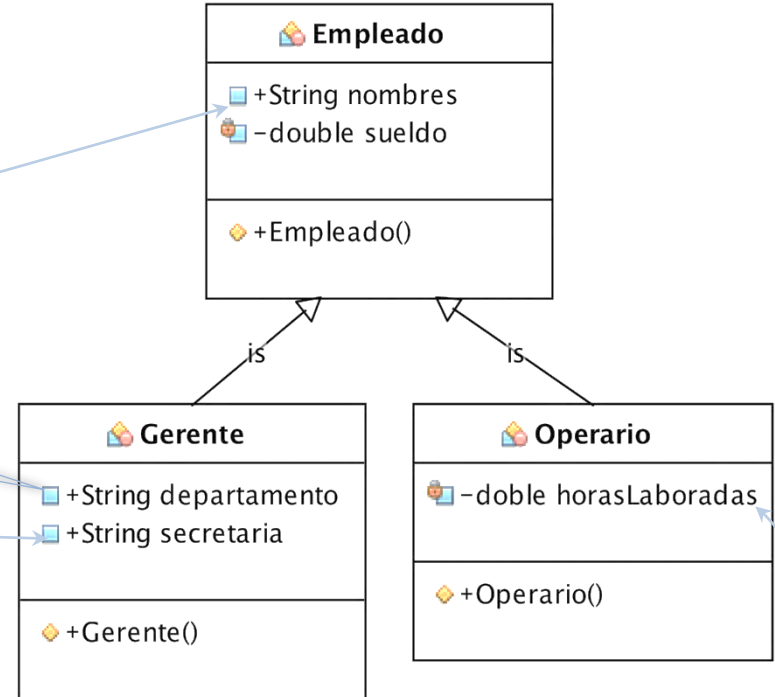
ESTUDIO DE CASO (1) (ANÁLISIS)

Defina una jerarquía de herencia en la que se contemplen los **empleados** de una empresa (*operarios* y *gerentes*), guarde los nombres y sueldo; para los gerentes como un tipo especial de empleados almacene el departamento que dirige y el nombre de su secretaria, para operarios, horas laboradas. Realizar la especificación UML con las clases implicadas.



ESTUDIO DE CASO (1) (ANÁLISIS)

Defina una jerarquía de herencia en la que se contemplen los **empleados** de una empresa (**operarios** y **gerentes**), guarde los nombres y sueldo; para los gerentes como un tipo especial de empleados almacene el departamento que dirige y el nombre de su secretaria, para operarios, horas laboradas. Realizar la especificación UML con las clases implicadas.



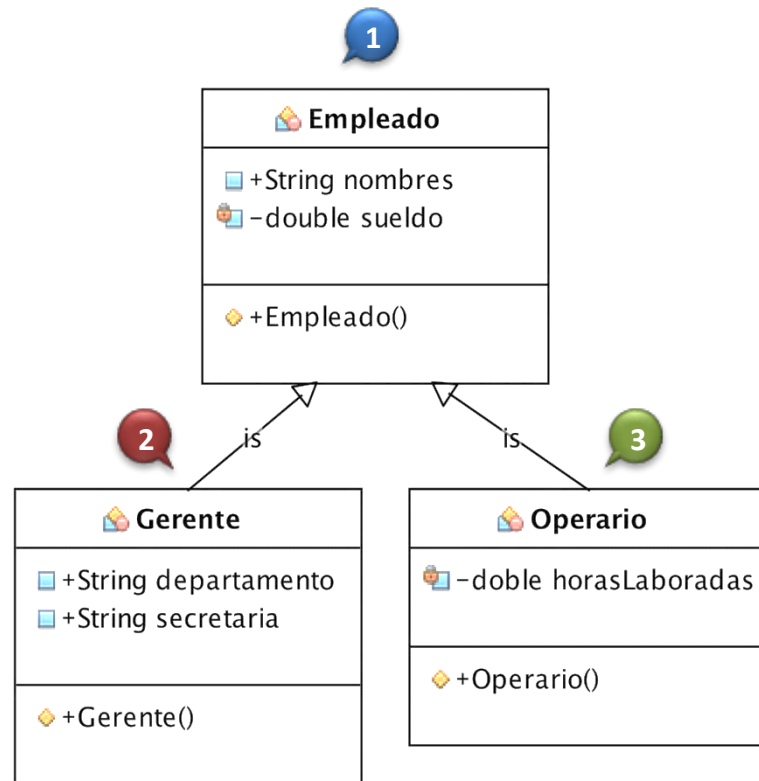
```

1 public class Empleado {
    public String nombres;
    private double sueldo;
    public Empleado() { }
}

2 class Gerente extends Empleado {
    public String departamento;
    public String secretaria;
    public Gerente() { }
}

3 class Operario extends Empleado {
    private double horasLaboradas;
    public Operario() { }
}

```




```

public class Empleado {
    public String nombres;
    private double sueldo;
    public Empleado() { }
    public Empleado(String nombres, double sueldo) {
        this.nombres = nombres;
        this.sueldo = sueldo;
    }
    public double getSueldo() {
        return sueldo;
    }
}

class Gerente extends Empleado {
    public String departamento;
    public String secretaria;
    public Gerente() { }
    public Gerente(String departamento, String secretaria) {
        this.departamento = departamento;
        this.secretaria = secretaria;
    }
    public Gerente(String departamento, String secretaria,
        String nombres, double sueldo) {
        super(nombres, sueldo);
        this.departamento = departamento;
        this.secretaria = secretaria;
    }
    @Override
    public String toString(){
        return String.format("Nombres: %s \nSueldo: %.2f \n"
            + "Departamento: %s \nSecretaria: %s"
            ,this.nombres, this.getSueldo(),
            this.departamento, this.secretaria);
    }
}

```



```

class Operario extends Empleado {
    private double horasLaboradas;
    public Operario() { }
    public Operario(double horasLaboradas) {
        this.horasLaboradas = horasLaboradas;
    }
    public Operario(double horasLaboradas, String nombres,
        double sueldo) {
        super(nombres, sueldo);
        this.horasLaboradas = horasLaboradas;
    }
    @Override
    public String toString(){
        return String.format("Nombres: %s \nSueldo: %.2f \n"
            + "Horas Laboradas: %.2f"
            ,this.nombres, this.getSueldo(),
            this.horasLaboradas);
    }
}

class TestEmpleadoGerentOperario{
    public static void main(String[] args) {
        Gerente gerente1 = new Gerente("Comput. y Elect.",
            "Laydi Sanmartin", "Juan Carrion", 100);
        Operario operario1 = new Operario(46.50,
            "Maria Vilareal", 200);
        System.out.println("GERENTE 1\n=====\n"+gerente1);
        System.out.println("\nOPERARIO 1\n=====\n"+operario1);
    }
}

```

Output - NomProyecto (run)

GERENTE 1	OPERARIO 1
=====	=====
Nombres: Juan Carrion	Nombres: Maria Vilareal
Sueldo: 100.00	Sueldo: 200.00
Departamento: Comput. y Elect.	Horas Laboradas: 46.50
Secretaria: Laydi Sanmartin	



ESTUDIO DE CASO (2)



Se desea realizar una aplicación que permita a los mecánicos de un garaje **registrar, consultar y actualizar** los trabajos (*reparaciones y revisiones*) que han sido realizados o que están en proceso de realización en el garaje.

Cada trabajo se identifica unívocamente por su **“identificador de trabajo”**. El **“identificador de trabajo”** es un número que se asocia con el trabajo en el momento que se registra. El primer trabajo registrado tendrá el identificador **0**, el segundo el **1** y así sucesivamente. Los trabajos incluyen una pequeña descripción de la reparación o revisión a realizar. Al crear un trabajo el número de horas es **0**. El número de horas ira aumentando a medida que los mecánicos van dedicando tiempo a realizar la reparación o la revisión. Cuando el trabajo se ha finalizado se marca como **“finalizado”** y el número de hora no puede volver a cambiarse.

Las reparaciones incluyen el precio del material utilizado (*pieza o pintura*): Al registrar una reparación el precio del material es **0** y va aumentando a medida que los mecánicos van utilizando material en la reparación. Una vez que la reparación se mancha como **“finalizada”** no se puede cambiar el precio del material utilizado.

El precio a cobrar para cada trabajo se compone de dos partes, **una fija** que resulta de multiplicar el número de horas empleadas por **\$30**; y la **variable** que, dependiendo del tipo de trabajo, su coste varía de la siguiente manera:

- ✓ **Reparación mecánica:** su precio se calcula como fijo más el *coste* del material multiplicado por el **1.1**
- ✓ **Reparación de chapas y pintura:** su precio se calcula como fijo más el *coste* del material multiplicado por **1.3**
- ✓ **Revisión:** su precio se calcula como fijo más un extra de **\$20** independiente del número de horas.

El garaje se ha marcado unos plazos máximos en los que deberían estar terminados los trabajos. Estos plazos son: **14 días** para las *reparaciones mecánicas*, **21 días** para las *reparaciones de chapas y pintura*, y **7** para las *revisiones*.

La aplicación se gestionará mediante una aplicación basada en menús, el cual permitirá: Registrar trabajos, Aumenta horas, Aumenta coste de piezas, Finaliza trabajo, Muestra trabajo, Consulta plazo y Muestra todos.

ESTUDIO DE CASO (2) (ANÁLISIS)



Se desea realizar una aplicación que permita a los mecánicos de un garaje **registrar, consultar y actualizar** los **trabajos** (**reparaciones** y **revisiones**) que han sido realizados o que están en proceso de realización en el garaje.

Cada trabajo se identifica unívocamente por su “identificador de trabajo”. El “*identificador de trabajo*” es un número que se asocia con el trabajo en el momento que se registra. El primer trabajo registrado tendrá el identificador **0**, el segundo el **1** y así sucesivamente. Los trabajos incluyen una pequeña descripción de la reparación o revisión a realizar. Al crear un trabajo el número de horas es **0**. El número de horas ira aumentando a medida que los mecánicos van dedicando tiempo a realizar la reparación o la revisión. Cuando el trabajo se ha finalizado se marca como “finalizado” y el número de hora no puede volver a cambiarse. Las reparaciones incluyen el precio del material utilizado (*pieza o pintura*): Al registrar una reparación el precio del material es 0 y va aumentando a medida que los mecánicos van utilizando material en la reparación. Una vez que la reparación se manca como “finalizada” (estado) no se puede cambiar el precio del material utilizado.

El precio a cobrar para cada trabajo se compone de dos partes; una fija que resulta de multiplicar el número de horas empleadas por \$30; y la variable que, dependiendo del tipo de trabajo, su coste varía de la siguiente manera:

- ✓ **Reparación mecánica**: su precio se calcula como fijo más el coste del material multiplicado por el 1.1
- ✓ **Reparación de chapas y pintura**: su precio se calcula como fijo más el coste del material multiplicado por 1.3
- ✓ **Revisión**: su precio se calcula como fijo más un extra de \$20 independiente del número de horas.

El garaje se ha marcado unos plazos máximos en los que deberían estar terminados los trabajos. Estos plazos son: **14** días para las reparaciones mecánicas, **21 días** para las reparaciones de chapas y pintura, y **7** para las revisiones.

La aplicación se gestionará mediante una aplicación basada en menús, el cual permitirá: Registrar trabajos, Aumenta horas, Aumenta coste de piezas, Finaliza trabajo, Muestra trabajo, Consulta plazo y Muestra todos.

ESTUDIO DE CASO (2) (ANÁLISIS)

Se desea realizar una aplicación que permita a los mecánicos de un garaje **registrar, consultar y actualizar** los **trabajos (reparaciones y revisiones)** que han sido realizados o que están en proceso de realización en el garaje.

Cada trabajo se identifica unívocamente por su "identificador de trabajo". El "identificador de trabajo" es un número que se asocia con el trabajo en el momento que se registra. El primer trabajo registrado tendrá el identificador 0, el segundo el 1 y así sucesivamente. Los trabajos incluyen una pequeña descripción de la reparación o revisión a realizar. Al crear un trabajo el número de horas es 0. El número de horas irá aumentando a medida que los mecánicos van dedicando tiempo a realizar la reparación o la revisión. Cuando el trabajo se ha finalizado se marca como "finalizado" y el número de hora no puede volver a cambiarse.

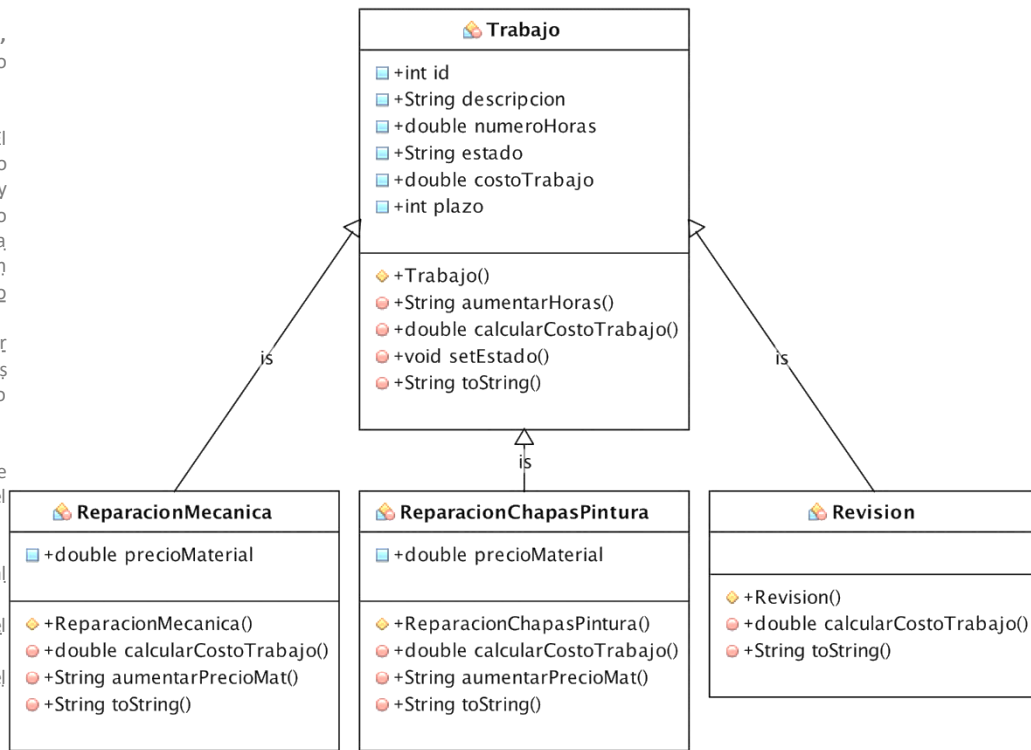
Las reparaciones incluyen el precio del material utilizado (pieza o pintura): Al registrar una reparación el precio del material es 0 y va aumentando a medida que los mecánicos van utilizando material en la reparación. Una vez que la reparación se marca como "finalizada" (estado) no se puede cambiar el precio del material utilizado.

El precio a cobrar para cada trabajo se compone de dos partes una fija que resulta de multiplicar el número de horas empleadas por \$30; y la variable que dependiendo del tipo de trabajo, su coste varía de la siguiente manera:

- ✓ **Reparación mecánica:** su precio se calcula como fijo más el coste del material multiplicado por el 1.1
- ✓ **Reparación de chapas y pintura:** su precio se calcula como fijo más el coste del material multiplicado por 1.3
- ✓ **Revisión:** su precio se calcula como fijo más un extra de \$20 independiente del número de horas.

El garaje se ha marcado unos plazos máximos en los que deberían estar terminados los trabajos. Estos plazos son: **14** días para las reparaciones mecánicas, **21 días** para las reparaciones de chapas y pintura, y **7** para las revisiones.

La aplicación se gestionará mediante una aplicación basada en menús, el cual permitirá: Registrar trabajos, Aumenta horas, Aumenta coste de piezas, Finaliza trabajo, Muestra trabajo, Consulta plazo y Muestra todos.





PROGRAMACIÓN ORIENTADA A OBJETOS - HERENCIA

curso de programación de algoritmos

@utpl, @xavicrip